B17 Testing and performance

Unit tests, Hypothesis, profiling and memory use.

B17-L1 Unit tests for data code that catch real bugs

Test edges on purpose, with tiny explicit inputs, one behaviour per test, and prove each test can fail.

```python
import pytest


def test_share_zero_total_is_none():
    assert share(3, 0) is None


@pytest.mark.parametrize(("part", "total", "expected"), [(1, 4, 0.25), (0, 5, 0.0), (5, 5, 1.0)])
def test_share_values(part, total, expected):
    assert share(part, total) == expected
```

A test worth writing	One that fails when the code is wrong. Break the code once to prove it.
Edge cases for data code	Empty input, missing values, zero denominators, one row, duplicate keys.
Compare DataFrames in tests	pd.testing.assert_frame_equal: values, dtypes, columns and index.
Compare floats in tests	pytest.approx(expected).

B17-L2 Property-based tests with Hypothesis

State a rule that must always hold, generate inputs from the domain the function promises, and let Hypothesis find and shrink the counterexample.

```python
from hypothesis import given, strategies as st

@given(st.lists(st.tuples(st.floats(-1e6, 1e6), st.floats(0, 10)), min_size=1))
def test_weighted_mean_within_range(pairs):
    values = [v for v, _ in pairs]
    weights = [w for _, w in pairs]
    m = weighted_mean(values, weights)
    assert m is None or min(values) - 1e-6 <= m <= max(values) + 1e-6
```

Property-based test	A rule checked against many generated inputs, with failing inputs shrunk to a minimal case.
Four data-code properties	Bounds, invariance, round trip, idempotence, grain, and agreement with a slow version.
st.floats() default	Includes nan and infinities. Bound it or pass allow_nan=False on purpose.
Hypothesis example database	.hypothesis/ replays past failures. Keep it out of Git.

B17-L3 Measure before you optimise

Profile to find the cost, time the change with perf_counter or timeit, keep setup out of the timing, and fix the biggest cost first.

```python
import cProfile, pstats
cProfile.run("main()", "profile.out")
```

```
pstats.Stats("profile.out").sort_stats("cumulative").print_stats(10)
```

Find where time goes	cProfile with pstats sorted by cumulative, or %prun in a notebook.
Time a small change	timeit.repeat or perf_counter. Report the minimum per call.
Measure memory	tracemalloc peak, and df.memory_usage(deep=True) for pandas.
The order of work	Profile, fix the biggest cost, re-measure, keep a test around the change.